krishnasree08@krishnasree08

GNU nano 7.2 active_jobs.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background job running. PID: %d\n", getpid());
        fflush(stdout);
        sleep(10);
        _exit(0);
    }

    printf("\nActive Job List\n");
    printf("Job ID: 1 | PID: %d | Status: Running\n", pid);
    printf("Parent PID: %d\n", getpid());

    waitpid(pid, NULL, 0);

    printf("Job completed and status updated.\n");
    return 0;
}
```

```
krishnasree08@krishnasree08:~/ossp_session15_16$ mkdir -p ~/ossp_session17_1
8
cd ~/ossp_session17_18
krishnasree08@krishnasree08:~/ossp_session17_18$ nano active_jobs.c
krishnasree08@krishnasree08:~/ossp_session17_18$ gcc -Wall -Wextra active_jo
bs.c -o active_jobs
./active_jobs

Active Job List
Job ID: 1 | PID: 652 | Status: Running
Parent PID: 651
Background job running. PID: 652


Job completed and status updated.
krishnasree08@krishnasree08:~/ossp_session17_18$
krishnasree08@krishnasree08:~/ossp_session17_18$
krishnasree08@krishnasree08:~/ossp_session17_18$ nano foreground_job.c
krishnasree08@krishnasree08:~/ossp_session17_18$ gcc -Wall -Wextra foregroun
d_job.c -o foreground_job
./foreground_job
Parent waiting for foreground job...
Foreground job started. PID: 660
Job running: step 1
Job running: step 2
Job running: step 3
Job running: step 4
Job running: step 5
Foreground job completed.
Foreground job finished.
```

```
krishnasree08@krishnasree08:~/ossp_session17_18$ nano signal_demo.c
krishnasree08@krishnasree08:~/ossp_session17_18$ gcc -Wall -Wextra signal_de
mo.c -o signal_demo
./signal_demo
Child process started. PID: 670
Parent sending SIGTERM to child 670
Child terminated by signal 15
Signal demonstration completed.
krishnasree08@krishnasree08:~/ossp_session17_18$ nano resume_job.c
krishnasree08@krishnasree08:~/ossp_session17_18$ gcc -Wall -Wextra resume_jo
b.c -o resume_job
./resume_job
Child job started. PID: 680
Job progress: 1/5
Job progress: 2/5
Parent stopping child job...
Job stopped. Sending SIGCONT...
Job progress: 3/5
Job progress: 4/5
Job progress: 5/5
Job resumed and completed.
krishnasree08@krishnasree08:~/ossp_session17_18$ cd ~/ossp_session17_18

gcc -Wall -Wextra active_jobs.c -o active_jobs
gcc -Wall -Wextra foreground_job.c -o foreground_job
gcc -Wall -Wextra signal_demo.c -o signal_demo
gcc -Wall -Wextra resume_job.c -o resume_job
krishnasree08@krishnasree08:~/ossp_session17_18$
```

OSSP Sessions 17 & 18 — Overall Result

Thus, the programs were implemented to demonstrate active job listing, process status monitoring, foreground process execution, signal delivery, process control, and resumption of stopped jobs using system calls such as fork(), waitpid(), kill(), and pause(). The programs also demonstrate process synchronization, job status updates, and background process recovery.